

LAB 12

NAME: Hajra Sarwar

REG NO: 2023-BSE-022

CLASS: BSE-5A

SUBJECT: CC

LAB TITLE: Terraform Provisioners, Modules & Nginx Reverse Proxy/Load Balancer

TASK:

Task 0: Lab Setup (Codespace & GH CLI)

1. Create GitHub repository and Codespace

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $ gh codespace list
NAME              DISPLAY_NAME  REPOSITORY      BRANCH  STATE    CREATED_AT
fluffy-tribble-wrq756... hlofaam      hajrasarwar11/CC... main*   Available about 6 days ago
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $
```

2. Connect to Codespace using SSH

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $ gh codespace ssh -c fluffy-tribble-wrq756x47vr25q6q
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

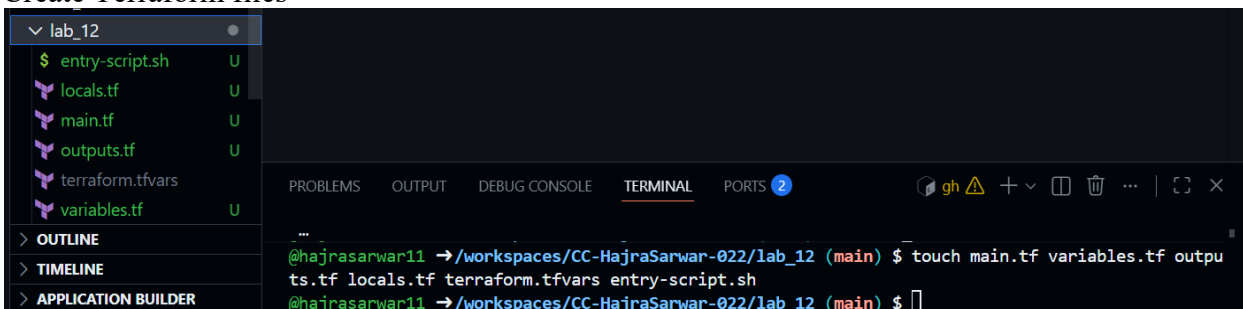
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

Task 1: Organize Terraform code into separate files

1. Create Lab12 directory

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $ cd lab_12
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $
```

2. Create Terraform files



The screenshot shows a code editor with a file explorer on the left displaying the contents of the 'lab_12' directory. The files listed are: entry-script.sh, locals.tf, main.tf, outputs.tf, terraform.tfvars, and variables.tf. The main editor area shows the terminal output for the command 'touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars entry-script.sh'.

```
lab_12
├── entry-script.sh
├── locals.tf
├── main.tf
├── outputs.tf
├── terraform.tfvars
└── variables.tf

@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ touch main.tf variables.tf outputs
ts.tf locals.tf terraform.tfvars entry-script.sh
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $
```

3. Add variables configuration

```
lab_12 > variables.tf > ...
1 variable "vpc_cidr_block" {}
2 variable "subnet_cidr_block" {}
3 variable "availability_zone" {}
4 variable "env_prefix" {}
5 variable "instance_type" {}
6 variable "public_key" {}
7 variable "private_key" {}
```

4. Add outputs configuration

```
lab_12 > outputs.tf > output "aws_instance_public_ip"
1 output "aws_instance_public_ip" {
2     value = aws_instance.myapp-server.public_ip
3 }
```

5. Add locals configuration

```
lab_12 > locals.tf > locals
1 locals {
2     my_ip = "${chomp(data.http.my_ip.response_body)}/32"
3 }
4
5 data "http" "my_ip" {
6     url = "https://icanhazip.com"
7 }
```

6. Add terraform.tfvars values

```
lab_12 > terraform.tfvars > abc vpc_cidr_block
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 availability_zone = "me-central-1a"
4 env_prefix = "dev"
5 instance_type = "t3.micro"
6 public_key = "~/ssh/id_ed25519.pub"
7 private_key = "~/ssh/id_ed25519"
```

7. Add main Terraform configuration

```
lab_12 > main.tf > ...
1 provider "aws" {
2     shared_config_files = ["~/aws/config"]
3     shared_credentials_files = ["~/aws/credentials"]
4 }
5
6 resource "aws_vpc" "myapp_vpc" {
7     cidr_block = var.vpc_cidr_block
8     tags = {
9         Name = "${var.env_prefix}-vpc"
10     }
```

8. Add entry script for nginx installation

```
lab_12 > $ entry-script.sh
1  #!/bin/bash
2  set -e
3  yum update -y
4  yum install -y nginx
5  systemctl start nginx
6  systemctl enable nginx
```

9. Generate SSH key pair

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ gh codespace ssh -c fluffy-tribble-wrq7
56x47vr25q6q
Generating public/private ed25519 key pair.
Your identification has been saved in /home/vscode/.ssh/id_ed25519
Your public key has been saved in /home/vscode/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:G2m500FARasWQcwij/KuOAsi2ogVFxcB9BYOBorSAR4 vscode@codespaces-e2c930
The key's randomart image is:
+--[ED25519 256]--+
|.Eoo=.+*=oo
|oo.O.+ooo .
|+.. .+=.. o
|. . .+ *
|.O. S .
| o. o = .
|+ .. + .
|B= . .
|*oo.
+----[SHA256]-----+
```

10. Initialize Terraform

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ gh codespace ssh -c fluffy-tribble-wrq7
56x47vr25q6q
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

11. Apply Terraform configuration

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022 (main) $ gh codespace ssh -c fluffy-tribble-wrq7
56x47vr25q6q
aws_default_security_group.default_sg: Creation complete after 2s [id=sg-0d89ecd719a2188c4]
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-0bf6a80a83ff32cb8]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "40.172.232.148"
```

12. View Terraform output

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform output  
aws_instance_public_ip = "40.172.232.148"
```

13. Verify nginx in browser

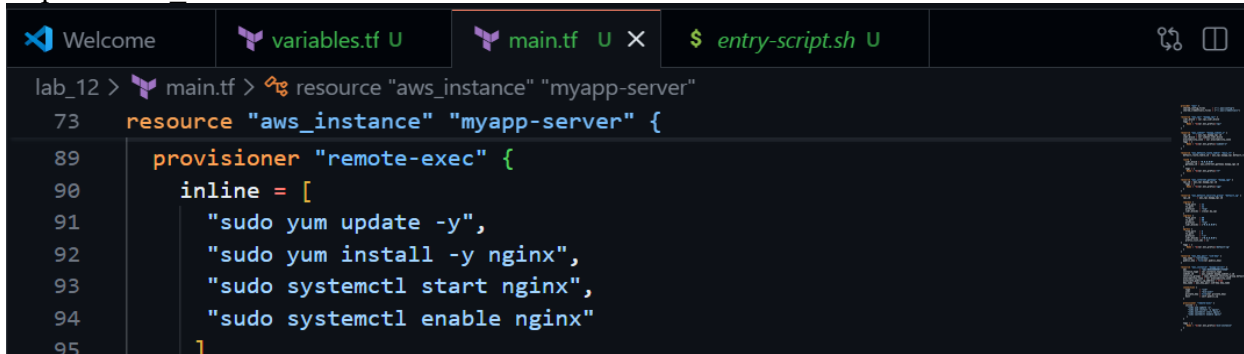


14. Destroy resources

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $ gh codespace ssh -c fluffy-tribble-wrq7  
56x47vrX25q6q  
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-019d88524243258c0]  
aws_key_pair.ssh-key: Destroying... [id=serverkey]  
aws_default_security_group.default_sg: Destroying... [id=sg-0d89ecd719a2188c4]  
aws_default_security_group.default_sg: Destruction complete after 0s  
aws_key_pair.ssh-key: Destruction complete after 1s  
aws_subnet.myapp_subnet_1: Destruction complete after 1s  
aws_vpc.myapp_vpc: Destroying... [id=vpc-0c052aa8e451842ee]  
aws_vpc.myapp_vpc: Destruction complete after 1s  
  
Destroy complete! Resources: 7 destroyed.
```

Task 2: Use remote-exec provisioner

1. Replace user data with remote-exec in main.tf



2. Apply Terraform

```
aws_instance.myapp-server (remote-exec): Complete!  
aws_instance.myapp-server (remote-exec): Created symlink /etc/systemd/system/multi-user.target.w  
ants/nginx.service → /usr/lib/systemd/system/nginx.service.  
aws_instance.myapp-server: Creation complete after 33s [id=i-0cc5fc7b949eac24c]  
  
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
aws_instance_public_ip = "40.172.113.34"
```

3. View Terraform output

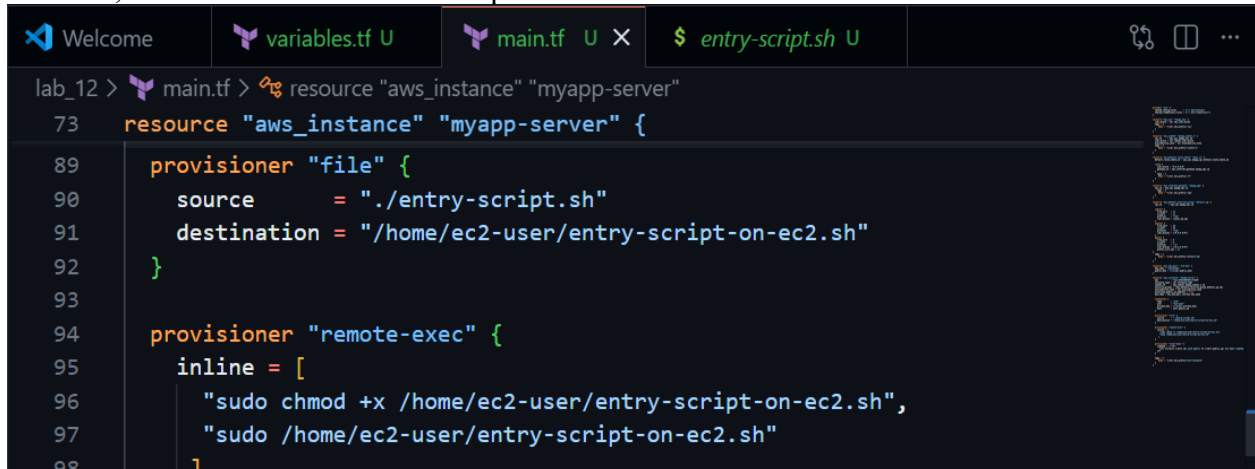
```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform output  
aws_instance_public_ip = "40.172.113.34"
```

4. Verify nginx in browser

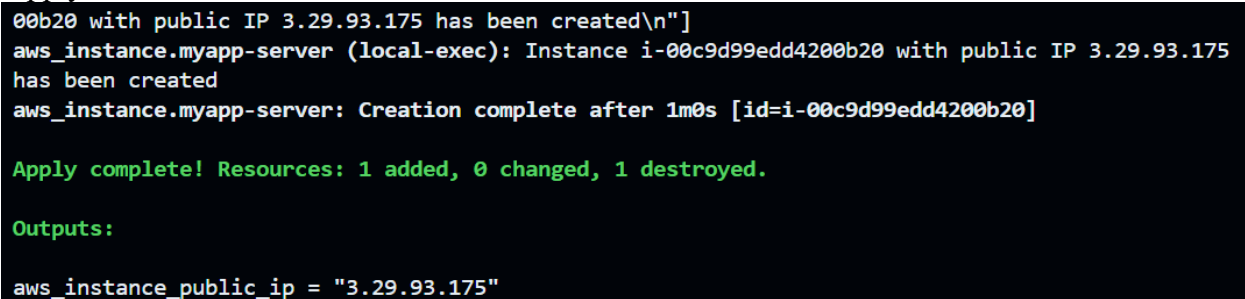


Task 3: Use file and local-exec provisioners

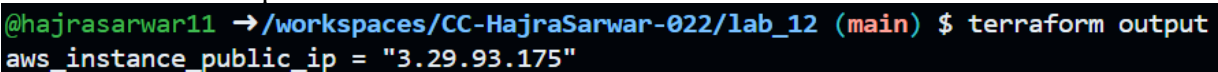
1. Add file, remote-exec and local-exec provisioners



2. Apply Terraform



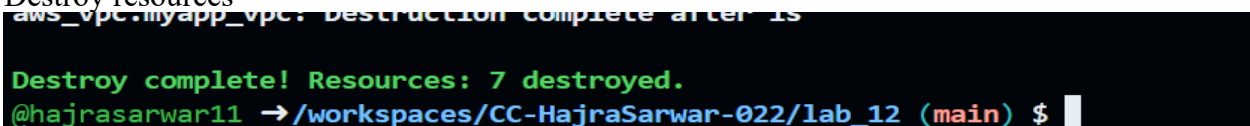
3. View Terraform output



4. Verify nginx in browser



5. Destroy resources



6. Restore user data method

```
lab_12 > main.tf > ...
73 resource "aws_instance" "myapp-server" {
82
83     user_data = file("./entry-script.sh")
84
85     tags = {
86         Name = "${var.env_prefix}-ec2-instance"
87     }
```

Task 4: Create Terraform modules (Subnet module)

1. Create subnet module structure

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $ tree lab_12/modules
lab_12/modules
├── subnet
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
2 directories, 3 files
```

2. Add subnet variables

```
lab_12 > modules > subnet > variables.tf > ...
1 variable "vpc_id" {}
2 variable "subnet_cidr_block" {}
3 variable "availability_zone" {}
4 variable "env_prefix" {}
5 variable "default_route_table_id" {}
```

3. Add subnet resources

```
lab_12 > modules > subnet > main.tf > ...
1 resource "aws_subnet" "myapp_subnet_1" {
2     vpc_id      = var.vpc_id
3     cidr_block  = var.subnet_cidr_block
4     availability_zone = var.availability_zone
5     map_public_ip_on_launch = true
6     tags = {
7         Name = "${var.env_prefix}-subnet-1"
8     }
```

4. Add subnet outputs

```
lab_12 > modules > subnet > outputs.tf > ...
1 output "subnet" {
2     value = aws_subnet.myapp_subnet_1
3 }
```

5. Use subnet module in root main.tf

```
lab_12 > main.tf > ...
54 resource "aws_instance" "myapp-server" {
55     ami            = "ami-05524d6658fcf35b6"
56     instance_type  = var.instance_type
57     subnet_id      = module.myapp-subnet.subnet.id
58     security_groups = [aws_default_security_group.default_sg.id]
59     availability_zone = var.availability_zone
60     associate_public_ip_address = true
61     key_name        = aws_key_pair.ssh-key.key_name
```

6. Initialize Terraform

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform init
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

7. Apply Terraform

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform apply -auto-approve
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-092e9efb0e6e0cfee]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws_instance_public_ip = "51.112.49.92"
```

8. View output

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform output
aws_instance_public_ip = "51.112.49.92"
```

9. Verify nginx in browser



Task 5: Create webserver module

1. Create webserver module structure

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ tree modules
modules
├── subnet
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
└── webserver
    ├── main.tf
    ├── outputs.tf
    └── variables.tf

3 directories, 6 files
```

2. Add webserver variables

```
lab_12 > modules > webserver > variables.tf > variable "vpc_id"
1  variable "env_prefix" {}
2  variable "instance_type" {}
3  variable "availability_zone" {}
4  variable "public_key" {}
```

3. Add webserver resources

```
main.tf lab_12 U main.tf ../webserver U X outputs.tf U
lab_12 > modules > webserver > main.tf > ...
1 resource "aws_security_group" "web_sg" {
2     vpc_id      = var.vpc_id
3     name        = "${var.env_prefix}-web-sg-${var.instance_suffix}"
4     description = "Security group for web server allowing HTTP, HTTPS and SSH"
5
6     ingress {
7         from_port = 22
8         to_port   = 22
9         protocol  = "tcp"
10        cidr_blocks = [var.my_ip]
```

4. Add webserver outputs

```
main.tf lab_12 U main.tf ../webserver U outputs.tf U X
lab_12 > modules > webserver > outputs.tf > ...
1 output "aws_instance" {
2     value = aws_instance.myapp-server
3 }
```

5. Use webserver module in root main.tf

```
main.tf lab_12 U X main.tf ../webserver U outputs.tf U
lab_12 > main.tf > ...
22 module "myapp-webserver" {
23     source      = "./modules/webserver"
24     env_prefix  = var.env_prefix
25     instance_type = var.instance_type
26     availability_zone = var.availability_zone
27     public_key  = var.public_key
28     my_ip       = local.my_ip
29     vpc_id      = aws_vpc.myapp_vpc.id
30     subnet_id   = module.myapp-subnet.subnet.id
31     script_path = "./entry-script.sh"
```

6. Update root outputs

```
main.tf lab_12 U outputs.tf U X main.tf ../webserver U
lab_12 > outputs.tf > output "webserver_public_ip"
1 output "webserver_public_ip" {
2     value = module.myapp-webserver.aws_instance.public_ip
3 }
4 output "aws_instance_public_ip" {
5     value = aws_instance.myapp-server.public_ip
6 }
```

7. Initialize Terraform

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform init
- Using previously-installed hashicorp/aws v6.28.0
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

8. Apply Terraform

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform apply -auto-approve
aws_instance.myapp-server: Destruction complete after 50s
aws_key_pair.ssh-key: Destroying... [id=serverkey]
aws_default_security_group.default_sg: Destroying... [id=sg-09ac2a296f58e305c]
aws_default_security_group.default_sg: Destruction complete after 0s
aws_key_pair.ssh-key: Destruction complete after 1s

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

Outputs:

webserver_public_ip = "3.29.67.212"
```

9. View output

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform output
webserver_public_ip = "3.29.67.212"
```

10. Verify nginx



11. Destroy resources

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform destroy
module.myapp-webserver.aws_security_group.web_sg: Destroying... [id=sg-00531ad45b7a1ffa9]
module.myapp-webserver.aws_key_pair.ssh-key: Destruction complete after 0s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 0s
module.myapp-webserver.aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-009464bdd9258365b]
aws_vpc.myapp_vpc: Destruction complete after 0s

Destroy complete! Resources: 7 destroyed.
```

Task 6: Configure HTTPS with self-signed certificates

1. Update entry script with SSL configuration

```
lab_12 > $ entry-script.sh
1  #!/bin/bash
2  set -e
3  yum update -y
4  yum install -y nginx
5  systemctl start nginx
6  systemctl enable nginx
7
8  # Create directories for SSL certificates if they don't exist
9  mkdir -p /etc/ssl/private
10 mkdir -p /etc/ssl/certs
```

2. Apply Terraform

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform apply -auto-approve
424889af21e]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ip = "3.29.63.209"
```

3. View output

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform output
webserver_public_ip = "3.29.63.209"
```

4. Verify HTTPS page



5. Verify HTTP to HTTPS redirect



Task 7: Configure Nginx as reverse proxy

1. Create Apache backend script

```
main.tf lab_12 U outputs.tf U entry-script.sh U $ apache.sh U X main.tf ..
```

```
lab_12 > $ apache.sh
1  #!/bin/bash
2  yum update -y
3  yum install httpd -y
4  systemctl start httpd
5  systemctl enable httpd
6  echo "<h1>Welcome to My Web Server</h1>" > /var/www/html/index.html
7  hostnamectl set-hostname myapp-webserver
8  echo "<h2>Hostname: $(hostname)</h2>" >> /var/www/html/index.html
9  TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
10  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

2. Add backend webserver module

```
main.tf lab_12 U X outputs.tf U entry-script.sh U $ apache.sh U main.tf ..
```

```
lab_12 > main.tf
35 module "myapp-web-1" {
36   source = "../modules/webserver"
37   env_prefix = var.env_prefix
38   instance_type = var.instance_type
39   availability_zone = var.availability_zone
```

3. Update outputs

```
lab_12 > outputs.tf > ...
1  output "webserver_public_ip" {
2    value = module.myapp-webserver.aws_instance.public_ip
3  }
4
5  output "aws_web-1_public_ip" {
6    value = module.myapp-web-1.aws_instance.public_ip
7  }
```

4. Apply Terraform

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform apply -auto-approve
module.myapp-web-1.aws_instance.myapp-server: Creation complete after 12s [id=i-0e678ed6bf61983e3]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:

aws_web-1_public_ip = "51.112.180.9"
webserver_public_ip = "3.29.63.209"
```

5. View outputs

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform output
aws_web-1_public_ip = "51.112.180.9"
webserver_public_ip = "3.29.63.209"
```

6. SSH into nginx server

```
❖ @hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_12 (main) $ ssh ec2-user@3.29.63.209
$ apache.sh U The authenticity of host '3.29.63.209 (3.29.63.209)' can't be established.
$ entry-script.sh U ED25519 key fingerprint is SHA256:+HDnhEnmIglvI+8Ktqj4VvCbwlK2eYhk6waNMW60kCI.
locals.tf U This key is not known by any other names.
main.tf U Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
outputs.tf U Warning: Permanently added '3.29.63.209' (ED25519) to the list of known hosts.

terraform.tfstate A newer release of "Amazon Linux" is available.
terraform.tfstate.backup Version 2023.10.20260105:
terraform.tfvars Version 2023.10.20260120:
variables.tf U Version 2023.10.20260120:

> Lab_Exam Run "/usr/bin/dnf check-release-update" for full release and version update info

> UbuntuMachine
> .gitignore
> gh_2.86.0_linux_amd64.ta...
> MyED25519Key.pem

> OUTLINE
> TIMELINE
> APPLICATION BUILDER
```

7. Configure nginx reverse proxy

```
server {
    listen 443 ssl;
    server_name 3.29.63.209;
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        proxy_pass http://51.112.180.9;
        proxy_pass http://backend_servers;
    }
}
```

8. Restart nginx

```
[ec2-user@ip-10-0-10-65 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-65 ~]$
```

9. View nginx logs and SSL files

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ ssh ec2-user@3.29.63.209
[ec2-user@ip-10-0-10-65 ~]$ cat /var/log/nginx/error.log
2026/01/24 09:13:11 [notice] 3016#3016: using the "epoll" event method
2026/01/24 09:13:11 [notice] 3016#3016: nginx/1.28.0
2026/01/24 09:13:11 [notice] 3016#3016: OS: Linux 6.1.158-180.294.amzn2023.x86_64
2026/01/24 09:13:11 [notice] 3016#3016: getrlimit(RLIMIT_NOFILE): 65535:65535
2026/01/24 09:13:11 [notice] 3057#3057: start worker processes
2026/01/24 09:13:11 [notice] 3057#3057: start worker process 3059
2026/01/24 09:13:11 [notice] 3057#3057: start worker process 3060
2026/01/24 09:13:11 [emerg] 3630#3630: invalid number of arguments in "pid" directive in /

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ ssh ec2-user@3.29.63.209
[ec2-user@ip-10-0-10-65 ~]$ cat /var/log/nginx/access.log
149.40.208.87 - - [24/Jan/2026:09:15:21 +0000] "GET / HTTP/1.1" 200 615 "-" "Mozilla/5.0 (
Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safar
i/537.36" "-"
149.40.208.87 - - [24/Jan/2026:09:15:23 +0000] "GET /favicon.ico HTTP/1.1" 404 3650 "http:
//3.29.63.209/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/143.0.0.0 Safari/537.36" "-"
185.16.39.146 - - [24/Jan/2026:09:24:35 +0000] "GET / HTTP/1.1" 200 615 "-" "Wget" "-"
114.111.54.188 - - [24/Jan/2026:09:26:47 +0000] "POST /cgi-bin/.%2e.%2e.%2e.%2e.%2e.%
2e/%2e.%2e.%2e.%2e/bin/sh HTTP/1.1" 400 157 "-" "-" "-"
114.111.54.188 - - [24/Jan/2026:09:26:47 +0000] "POST /cgi-bin/%32%65%32%65/%32%65%32%
65/%32%65%32%65/%32%65%32%65/%32%65%32%65/%32%65%32%65/%32%65%32%65/bin/sh HTTP/
[ec2-user@ip-10-0-10-65 ~]$ cat /etc/nginx/mime.types
types {
application/A2L                                a2l;
application/AML                                aml;
application/andrew-inset                       ez;
application/ATF                                atf;
application/ATFX                                atfx;
application/ATXML                              atxml;
application/atom+xml                           atom;
application/atomcat+xml                       atomcat;
application/atomdeleted+xml                   atomdeleted;
[ec2-user@ip-10-0-10-65 ~]$ cat /etc/ssl/certs/selfsigned.crt
-----BEGIN CERTIFICATE-----
MIIDOzCCAiOgAwIBAgIUJQ0+B10orNqM2RXXPFryxza0iaEwDQYJKoZIhvcNAQEL
BQAwFjEUMBIGA1UEAwLMYy4yOS42My4yMDkwHhcNMjYwMTI0MDkxMzExWWhcNMjYw
MTI0MDkxMzExWjEwOTCCASAwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBANuhGjMoutKMJKK2MoKTW7tuRmcU2/23jBX+p1G
YSHFUWexJ5GTZHG69xVpSoya4Ru8INZrt4GNQ3eJfjmNFixpCp4sSI2f/o2SHq1H
z3QXQXdAfosTQRbFt+B1dp7ZtJhCtnM1VAAJPmtjvJGjJcZiQCsv8hirI7qZFXDC
wBwvGwRcFXJ3xqLnJTFxHnMzsdjdJs9MX4XDxk6cKAKxW6fvm+/IOFOFW3cdyrz
dQp1xP+LiD0vmIfw+CvzaYVJHj05EmUtACjwc35ViZ5t8eoDK//GjLUV5/m9K2+G
VA915MrqX4wFKUa7rFs93S09JfKH4pGAZxXAKzrK05QGCWECAwEAa0BgDB+MB0G
A1UdDgQWBRRgMDcBhCKdz2P7iW4Kq1QP07KHITAFBgNVHSMEGDAWBgRgMDcBhCKd
z2P7iW4Kq1QP07KHITAPBgNVHREECDAGhwQDHT/RMAkGA1UEwQCMAAwCwYDVROp
BAQDAgWgMBMGAlUdJQ0MMAoGCCSGAQUFBwMBMA0GCsGqGSIB3DQEBCwUAA4IBAQDJ
OX0uW4h1N22vKp0t91vo+7LUqWVMioGK5bODkF7LGczYi6negYcg4ApYx8jC5j8
3enXnwisy8V6UBe9j1zRrtwUBq3QX9R78E1bdsmN1Hyams9KnDgqtj6ZnunMadUv
uo6S/OFl1ToaLBZxLCGSNO16KcQawqhF3R42wEf04ycMpTmveWzQhXAVf01xw++hu
a3q011RKtCvriErRaVi6H9z+3K20x/6RvHVapiaYuEqWjSIEBXG7LEV992wm8Sk
MADp1w4h1N22vKp0t91vo+7LUqWVMioGK5bODkF7LGczYi6negYcg4ApYx8jC5j8
3enXnwisy8V6UBe9j1zRrtwUBq3QX9R78E1bdsmN1Hyams9KnDgqtj6ZnunMadUv
uo6S/OFl1ToaLBZxLCGSNO16KcQawqhF3R42wEf04ycMpTmveWzQhXAVf01xw++hu
a3q011RKtCvriErRaVi6H9z+3K20x/6RvHVapiaYuEqWjSIEBXG7LEV992wm8Sk
-----END CERTIFICATE-----

[ec2-user@ip-10-0-10-65 ~]$ sudo cat /etc/ssl/private/selfsigned.key
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEAAoIBAQDTboRozDrrSjCS
itjKcK1u7bkTHFNv9t4wV/qZRmEhxVFnlYeRk2YRuvCvAUjsmuEbvCDWa7eBjUN3
ix45jRYsaQqeLEiNn/6Nkh6pR890F0F3QH6LE0EWxbfgdXae2bSYQrZzNVQACT5r
Y7yRoyXGYkArL/IYqyO6mRVwmsAcL4FokXBVyd8ai5yUxcR5zM7HY3SbPTF+Fw8Z
QnCcCsVun75vWvDbThVt3Hco83UK7cT/i4g9L5iH8Prr82mLvR4zuR711QAo8HN+
-----END PRIVATE KEY-----
```

10. Verify reverse proxy in browser

← → ↻ 🏠 🔒 Not secure https://3.29.63.209

Welcome to My Web Server

Hostname: myapp-webserver

Private IP:

Public IP:

Public DNS:

Deployed via Terraform

Task 8: Configure Nginx as load balancer

1. Add second backend server

```
lab_12 > main.tf > ...
48 module "myapp-web-2" {
49     source = "../modules/webserver"
50     env_prefix = var.env_prefix
51     instance_type = var.instance_type
52     availability_zone = var.availability_zone
53     public_key = var.public_key
54     my_ip = local.my_ip
55     vpc_id = aws_vpc.myapp_vpc_id
```

2. Update outputs

```
lab_12 > outputs.tf > ...
9  output "aws_web-2_public_ip" {
10     value = module.myapp-web-2.aws_instance.public_ip
11 }
```

3. Apply Terraform

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform apply -auto-approve

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

aws_web-1_public_ip = "51.112.180.9"
aws_web-2_public_ip = "3.28.204.85"
webserver_public_ip = "3.29.63.209"
```

4. View outputs

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ ssh ec2-user@3.29.63.209

A newer release of "Amazon Linux" is available.
Version 2023.10.20260105:
Version 2023.10.20260120:
Version 2023.10.20260120:
Run "/usr/bin/dnf check-release-update" for full release and version update info

#_
~\  #####      Amazon Linux 2023
~~ \#####\
~~  \###|
~~   \#/  ____  https://aws.amazon.com/linux/amazon-linux-2023
~~    V~'  '->
~~~~
~~~. _./
~~  /  /
~~ /m/'

Last login: Sat Jan 24 10:47:07 2026 from 4.240.39.196
[ec2-user@ip-10-0-10-65 ~]$
```

5. Configure nginx load balancing

```
upstream backend_servers {
    server 51.112.180.9:80;
    server 3.28.204.85:80;
}

server {
    listen 443 ssl;
    server_name 3.29.63.209;
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        proxy_pass http://51.112.180.9:80;
        proxy_pass http://backend_servers;
    }
}
```

6. Restart nginx

```
[ec2-user@ip-10-0-10-65 ~]$ sudo systemctl restart nginx
[ec2-user@ip-10-0-10-65 ~]$
```

Task 9: Configure high availability with backup servers

1. Configure web-1 primary

```
o @hajrasarwar11 → /workspaces/CC-HajraSarwar-022 (main) $ ssh ec2-user@3.29.63.209
```

A newer release of "Amazon Linux" is available.

Version 2023.10.20260105:
Version 2023.10.20260120:
Version 2023.10.20260120:

Run "/usr/bin/dnf check-release-update" for full release and version update info

```
,      #_  
~\_   #####_       Amazon Linux 2023  
~~    \#####\  
~~~~   \|###|  
~~~~    \#/_____ https://aws.amazon.com/linux/amazon-linux-2023  
~~~~     V~' '->  
~~~~  
~~~~_. _/_/  
~~~~ _/_/_/  
~~~~ _/m/'
```

Last login: Sat Jan 24 12:02:44 2026 from 4.240.39.196

```
[ec2-user@ip-10-0-10-65 ~]$ █
```

2. Verify web-1 only



← ↻ Not secure https://3.29.30.34

Welcome to My Web Server

Hostname: myapp-webserver

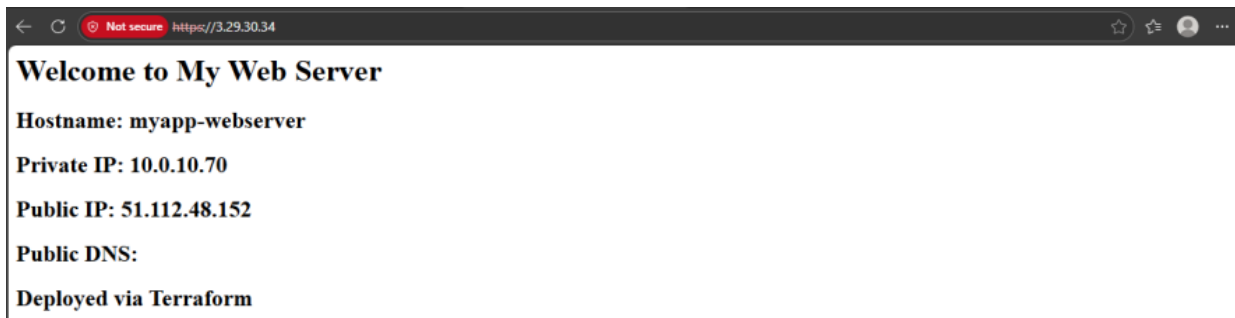
Private IP: 10.0.10.70

Public IP: 51.112.48.152

Public DNS:

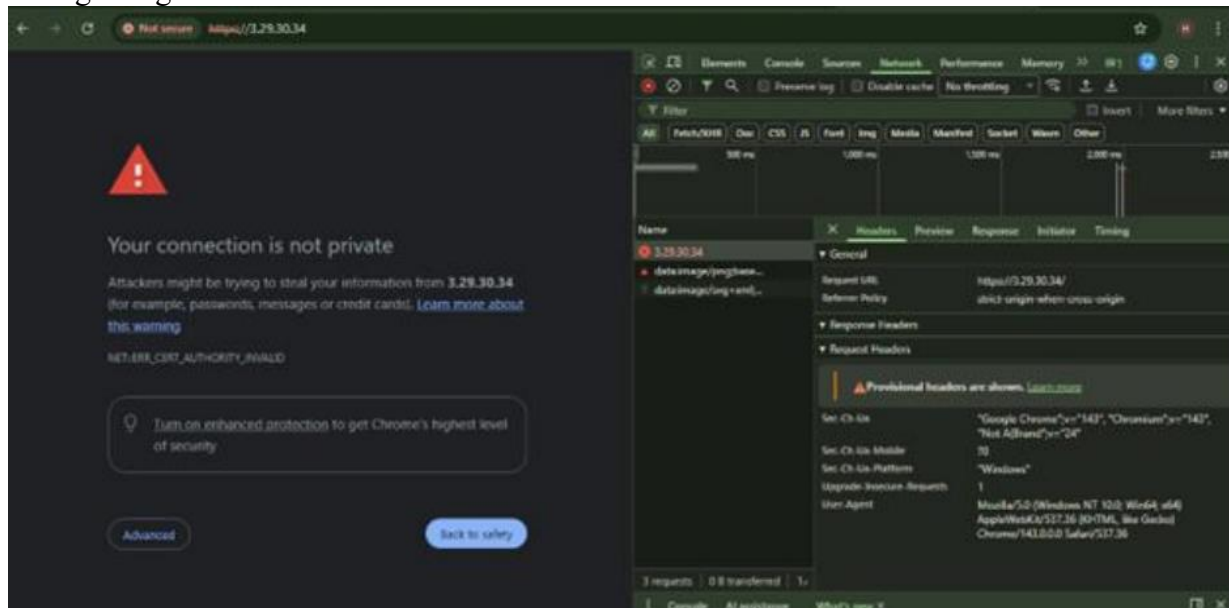
Deployed via Terraform

3. Configure web-2 primary

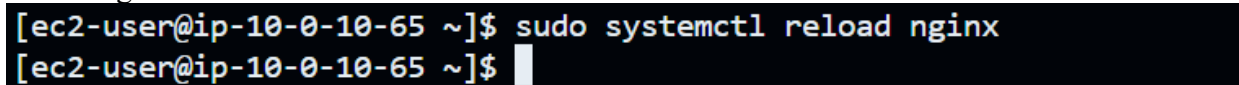


Task 10: Enable Nginx caching

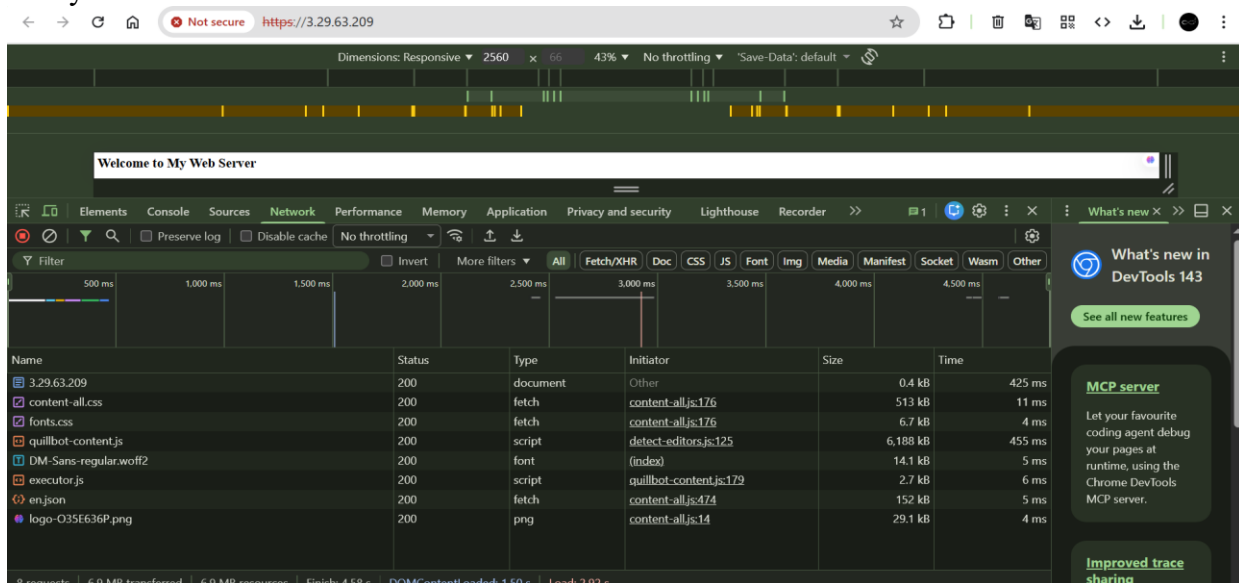
1. Configure nginx cache



2. Restart nginx



3. Verify cache MISS



Cleanup

1. Destroy all resources

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform destroy
- tags_all = {
  - "Name" = "dev-default-sg"
} -> null
- vpc_id = "vpc-0ebdc0b6cd81f4faa" -> null
# (1 unchanged attribute hidden)
}

Plan: 0 to add, 0 to change, 13 to destroy.

Changes to Outputs:
- aws_web-1_public_ip = "51.112.180.9" -> null
- aws_web-2_public_ip = "3.28.204.85" -> null
- webserver_public_ip = "3.29.63.209" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: 
```

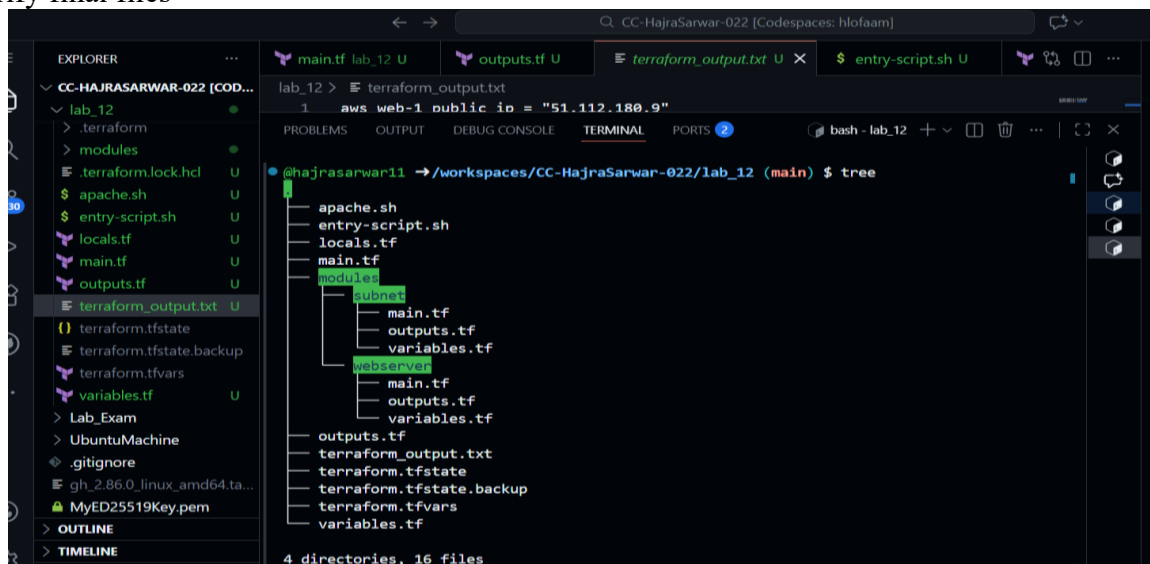
```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ terraform destroy
module.myapp-web-2.aws_instance.myapp-server: Destruction complete after 1m10s
module.myapp-web-2.aws_key_pair.ssh-key: Destroying... [id=dev-serverkey-2]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destroying... [id=subnet-0b3d482b94bd8991f]
module.myapp-web-2.aws_security_group.web_sg: Destroying... [id=sg-02ba95d5ac6d17798]
module.myapp-web-2.aws_key_pair.ssh-key: Destruction complete after 1s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 1s
module.myapp-web-2.aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0ebdc0b6cd81f4faa]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 13 destroyed.
```

2. Verify state

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 93,
  "lineage": "4458b676-6ac1-1909-111c-b68a95405807",
  "outputs": {},
  "resources": [],
  "check_results": null
}
```

3. Verify final files



```
CC-HajraSarwar-022 [Codespaces: hlofaam]
lab_12 > terraform_output.txt
1 aws_web-1_public_ip = "51.112.180.9"

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_12 (main) $ tree
.
├── apache.sh
├── entry-script.sh
├── locals.tf
├── main.tf
├── module
│   ├── subnet
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── webserver
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
├── outputs.tf
├── terraform_output.txt
├── terraform.tfstate
├── terraform.tfstate.backup
├── terraform.tfvars
└── variables.tf

4 directories, 16 files
```